

STeP Functional Emulation Model: Design Document

STeP Team

April 2026

1 Overview

The functional emulator computes *values* through a STeP dataflow graph without modeling timing, bandwidth, or hardware resource contention. Its purpose is to verify that a STeP graph produces the correct output tensor by comparing against a PyTorch reference implementation.

Contrast with cycle-accurate simulation. The Rust cycle-accurate simulator models per-cycle tile movement through compute units, on-chip buffers, and off-chip memory channels. The functional emulator abstracts all of this away: each operator is a pure function from input tensors to output tensors, evaluated once in topological order.

	Functional Emulator	Cycle-Accurate Sim
Language	Python (PyTorch)	Rust
Semantics	Value-level	Cycle-level
Output	Dense tensor	Dense tensor + timing trace
Speed	Milliseconds	Seconds–minutes
Use case	Graph correctness	Performance modeling

2 Tensor Representation

Every node produces a tiled tensor of shape

$$(\underbrace{s_0, s_1, \dots, s_{n-1}}_{\text{stream shape}}, \text{tile_r}, \text{tile_c})$$

where the last two dimensions are the spatial tile dimensions and the leading dimensions describe the tile grid (the *stream shape*).

Convention. Source nodes prepend a leading 1 to the stream shape, i.e. $s_0 = 1$, matching the STeP IR convention that every tile stream begins with a singleton batch dimension.

Multi-output nodes. `Broadcast` and `FlatPartition` produce a Python `list` of tensors rather than a single tensor. Downstream consumers reference a specific output index.

3 Execution Model

1. Compute a topological ordering of the graph (via NetworkX).
2. Walk nodes in order; dispatch each to its handler, which reads predecessor values from a dictionary keyed by instance ID and writes its own result back.
3. After all nodes are evaluated, the designated **OffChipStore** node’s input is *untiled* into a dense 2-D tensor and returned.

Untiling reverses the tiling: the leading singleton is stripped, tile grid dimensions are permuted so that row-tiles and col-tiles are adjacent, and the result is reshaped into (R, C) .

4 Operator Semantics

4.1 Source Nodes

LinearOffChipLoad. The underlying dense tensor is reshaped into a grid of $(R/\text{tile_r}) \times (C/\text{tile_c})$ tiles, flattened, and then indexed by a multi-dimensional stride mapping:

$$\text{linear_idx}(\mathbf{i}) = \sum_d i_d \cdot \text{stride}_d$$

This produces a tensor of shape $(\text{out_shape_tiled}, \text{tile_r}, \text{tile_c})$, to which a leading 1 is prepended.

LinearOffChipLoadRef. Same tiling as above, but the tile tensor is broadcast over the outer stream dimensions of a *reference* input, enabling weight reuse across batches.

SelectGen. Returns the raw multi-hot selection tensor (used by **FlatPartition/FlatReassemble** for MoE routing).

4.2 Compute Nodes

UnaryMap(f, x). Applies f element-wise to the full tiled tensor. Supported functions and their semantics:

Function	Semantics
Square	x^2
Silu	$x \cdot \sigma(x)$
Exp	e^x
Rsqrt	$1/\sqrt{x}$
Pow2	2^x
MulImmediate(c)	$x \cdot c$
AddImmediate(c)	$x + c$
SubImmediate(c)	$x - c$
RowWiseSum	$\sum_j x_{ij}$ (reduce last dim, keep dim)

BinaryMap(f, a, b). Applies f element-wise (with broadcasting) to two tiled tensors. Supported: Matmul (ab or $ab^\top$), Add, Mul, Div.

BinaryMapAccum(f, a, b, rank). Applies the map function f tile-wise, then sum-reduces over the last rank stream dimensions (dimension -3 relative to tile dims). Used for tiled matrix multiplication with accumulation:

$$C_{ij} = \sum_k f(A_{ik}, B_{kj})$$

Accum(f, x, rank). Reduces the last rank stream dimensions of x using function f :

Function	Semantics
Add	Sum-reduce along dim -3
Mul	Product-reduce along dim -3
RetileRow	Concatenate tiles vertically: $(\dots, K, t_r, t_c) \rightarrow (\dots, K \cdot t_r, t_c)$
RetileCol	Concatenate tiles horizontally: $(\dots, K, t_r, t_c) \rightarrow (\dots, t_r, K \cdot t_c)$

4.3 Structural Nodes

Broadcast(x, n). Duplicates the tensor n times; returns a list of n identical references. Inserted by `infer_broadcast` when one output feeds multiple consumers.

RepeatStatic(x, k). Inserts a new stream dimension of size k at position -3 (before tile dims) and expands. Used to replicate a reduced value (e.g., row-wise statistics) across k column tiles.

Bufferize / **Streamify**. **Bufferize** is an identity on values (it marks a buffering boundary in the timing model). **Streamify** optionally inserts and expands new stream dimensions with given repeat factors, broadcasting buffered values back into a stream.

Flatten($\text{min_rank}, \text{max_rank}$). Merges stream dimensions between min_rank and max_rank (counting from the right) into a single dimension.

Reshape($\text{chunk}, \text{rank}, \text{add_outer}$). Splits the stream dimension at position rank into $(\lceil D/\text{chunk} \rceil, \text{chunk})$, with optional zero-padding and an optional outer singleton dimension.

RetileStreamify($\text{chunk}, \text{split_row}/\text{split_col}$). Splits the tile's row or column dimension into chunks, merging the chunk count into the preceding stream dimension. Optional `filter_mask` removes zero-padded chunks.

Promote / **PromoteOuter**. Insert a singleton stream dimension at a specified rank (Promote) or at the outermost position (PromoteOuter).

ExpandRef / **RepeatRef**. Broadcast the input's stream dimensions to match a reference tensor's stream shape.

4.4 Routing Nodes (MoE)

FlatPartition(x, ctrl, n). Partitions a flat tile stream into n expert streams using a multi-hot selection tensor from `SelectGen`. For each expert i , selects tiles where `ctrl[:, i] > 0`. Returns a list of n tensors.

FlatReassemble($\{x_i\}$, **ctrl**). Inverse of partition: iterates over all token positions, reads from expert streams using per-expert pointers guided by the multi-hot mask, and stacks results. Output shape follows the control stream convention with a leading 1.

5 Validation

The validation harness (`validate_functional.py`) compares the functional emulator’s output against PyTorch reference implementations for all 14 seed kernels across 55 presets. The comparison uses relative error:

$$\text{rel_err} = \frac{\|y_{\text{sim}} - y_{\text{ref}}\|_{\infty}}{\|y_{\text{ref}}\|_{\infty} + 10^{-12}} < 10^{-5}$$

All 117 kernel×preset validations pass (as of April 2026).